

FENIXCMS_5

FASE 5 - PRODUCCIÓN REAL: FUENTE ÚNICA DE VERDAD, FALLBACKS CERRADOS Y PERSISTENCIA

INICIO FASE 5

DONDE SE EJECUTA:

AI STUDIO / repositorio fenixcmssl-spec/moreno

VALIDACIÓN:

ORDENADOR DEBIAN

INTEGRACIÓN:

VPS DE STAGING -> VPS PRODUCCIÓN

PRIORIDAD:

CRÍTICA - BLOQUEADOR DIRECTO PARA PRODUCCIÓN.

1. OBJETIVO MÁXIMO

La auditoría actual de main muestra que FenixCMS tiene PostgreSQL, aislamiento multi-tenant, PayPal server-side y el checkout de tienda FENIXCMS_4, pero todavía conserva rutas que pueden recurrir a datos demo, memoria o valores por defecto.

La regla de FENIXCMS_5 es:

****EN PRODUCCIÓN, POSTGRESQL + PROVEEDOR DE PAGO REAL + STORAGE PERSISTENTE SON LA ÚNICA FUENTE DE VERDAD.****

No puede existir una segunda realidad formada por:

- INITIAL_TENANTS
- INITIAL_PRODUCTS
- INITIAL_PLANS
- INITIAL_APPLICATIONS
- INITIAL_LICENSES
- INITIAL_THEMES
- INITIAL_PLUGINS
- FALLBACK_PAYMENTS
- inMemoryUserStore
- MEMORY_WEBHOOK_EVENTS
- tenant_demo
- usuarios ficticios
- pagos ficticios
- métricas ficticias
- valores monetarios por defecto
- storage que devuelve éxito aunque no haya guardado el archivo.

2. HALLAZGO MÁS IMPORTANTE DE LA AUDITORÍA

Archivo principal:

lib/services/payment.service.ts

La función PaymentService.createSaaSCheckoutSession() actualmente:

1. intenta buscar Application;
2. si falla o no encuentra el registro, puede usar INITIAL_APPLICATIONS;
3. intenta buscar Plan;
4. si falla o no lo encuentra, puede usar INITIAL_PLANS;
5. calcula importe desde el fallback;
6. construye tenantId;
7. llama a ensureTenantExists() antes de disponer de un pago capturado;
8. crea un Payment PENDING.

Esto es incompatible con producción.

La compra SaaS debe ser:

Application real

-> Plan real

-> CheckoutSession
-> PayPal Order
-> aprobación
-> Capture COMPLETED
-> provisioning
-> Tenant

Nunca:

Checkout
-> fallback INITIAL_*
-> ensureTenantExists()
-> Tenant.

3. REGLA ABSOLUTA

NO PAYMENT CAPTURED = NO PROVISIONING

Y además:

NO DATABASE CONFIRMATION = NO CHECKOUT COMPLETION

NO DATABASE = NO FAKE DATA

NO STORAGE WRITE = NO SUCCESS

NO VERIFIED WEBHOOK = NO PAYMENT UPDATE

4. FASE 5A - CERRAR PaymentService

Archivo:

lib/services/payment.service.ts

Cambios obligatorios

Eliminar de la ruta comercial:

- INITIAL_APPLICATIONS;
- INITIAL_PLANS;
- INITIAL_TENANTS;
- FALLBACK_PAYMENTS;
- precios por defecto;
- moneda por defecto si el plan no está disponible;
- creación anticipada de Tenant.

Application

Si PostgreSQL responde:

- buscar Application;
- si no existe -> 404/422;
- si existe -> continuar.

Si PostgreSQL falla:

- 503;
- no usar fallback.

Plan

Si PostgreSQL responde:

- buscar Plan;
- comprobar que pertenece a Application;
- comprobar activo;
- comprobar billingPeriod.

Si PostgreSQL falla:

- 503;
- no INITIAL_PLANS.

5. FASE 5B - CHECKOUT NO CREA TENANT

Eliminar:

ensureTenantExists(...)

del camino create checkout.

Create checkout solo puede crear:

- CheckoutSession;
- Payment PENDING si el diseño lo necesita;
- PayPal Order.

El Tenant definitivo se crea solamente dentro del provisioning posterior a Capture COMPLETED.

La regla arquitectónica es:

CREATE CHECKOUT != CREATE TENANT

6. FASE 5C - CHECKOUTSESSION COMO FUENTE DEL PEDIDO

CheckoutSession debe contener:

- applicationId;
- planId;
- billingPeriod;
- amountExpected;
- currencyExpected;
- customerName;
- customerEmail;
- tenantName;
- tenantSlug;
- billingAddress;
- paypalOrderId;
- paypalCaptureId;
- status;
- expiresAt;
- paymentId;
- tenantId;
- timestamps.

El navegador no puede cambiar estos datos durante capture.

7. FASE 5D - ÚNICA RUTA DE PROVISIONING

Crear o consolidar:

lib/services/saas-provisioning.service.ts

Método conceptual:

provisionCapturedCheckout(checkoutSessionId, verifiedCapture)

La única responsabilidad es convertir una CheckoutSession pagada y validada en:

- Payment;
- Tenant;
- User;
- Membership OWNER;
- License;
- Subscription;
- Invoice;
- Domain;
- CheckoutSession COMPLETED.

Ningún otro servicio debe crear Tenant por separado como parte del checkout.

8. FASE 5E - PROVISIONING ATÓMICO

Después de validar PayPal:

BEGIN TRANSACTION

1. Releer CheckoutSession.
2. Comprobar estado.
3. Comprobar que no está consumida.
4. Comprobar captureId único.
5. Crear Payment.
6. Crear/reutilizar User de forma segura.
7. Crear Tenant.
8. Crear Membership OWNER.
9. Crear License.
10. Crear Subscription.
11. Crear Invoice.
12. Crear dominio de sistema.
13. Marcar CheckoutSession COMPLETED.
14. Guardar consumedAt.
15. COMMIT.

Si falla una etapa:

ROLLBACK.

Nunca devolver success=true con recursos parciales.

9. FASE 5F - PAYMENT FAIL-CLOSED

Eliminar patrones como:

DB update failed

-> payment.status = COMPLETED en memoria

-> success

La única confirmación real es PostgreSQL.

Si falla la actualización:

- operación recuperable;
- no success final;
- no crear recursos duplicados;
- permitir reconciliation.

10. FASE 5G - AUTH FAIL-CLOSED

Archivo:

lib/services/auth.service.ts

El login de producción ya tiene una rama PostgreSQL-only, pero otras operaciones aún tienen fallback en memoria.

Revisar y corregir:

- getUserById;
- getUserByEmail;
- listUsers;
- updateUser;
- deleteUser;
- changePassword;
- cualquier función administrativa.

En NODE_ENV=production:

PostgreSQL error -> ERROR/503.

Nunca usar inMemoryUserStore para representar el estado real de usuarios.

Mantener fallback solo para tests/dev controlados y nunca como una rama alcanzable desde producción.

11. FASE 5H - ELIMINAR CREDENCIALES MAESTRAS

Auditar:

- lib/services/auth.service.ts
- lib/auth/*
- app/api/auth/*
- components/*

Prohibido:

- password === '...';
- admin123;
- Patricia1980@;
- fenix2026;
- cuentas maestras predefinidas como backdoor.

Crear, si es necesario, un bootstrap controlado del primer SUPER_ADMIN mediante variables de entorno y hash.

No guardar el secreto en Git.

12. FASE 5I - SUPER ADMIN SIN DATOS FICTICIOS

Archivo:

lib/services/super-admin.service.ts

Actualmente existen fallbacks para:

- INITIAL_TENANTS;
- INITIAL_LICENSES;
- INITIAL_PLANS;
- INITIAL_APPLICATIONS;
- INITIAL_THEMES;
- INITIAL_PLUGINS;
- pagos demo;
- métricas demo;
- usuarios demo.

En producción:

DB vacía -> 0 / [] / empty state.

DB caída -> 503 / unavailable.

Nunca:

DB caída -> INITIAL_*

13. FASE 5J - DIFERENCIAR DB VACÍA DE DB CAÍDA

Caso A:

PostgreSQL responde y devuelve 0 registros.

Resultado:

- 0 tenants;
- 0 licenses;
- 0 payments;
- 0 subscriptions.

Caso B:

PostgreSQL no responde.

Resultado:

- error de dependencia;
- readiness 503;
- no datos demo;
- no fake success.

14. FASE 5K - SUPER ADMIN METRICS

Eliminar:

```
catch() => INITIAL_TENANTS.length)
catch() => 3)
catch() => 4)
catch() => INITIAL_LICENSES)
```

En producción, un fallo de PostgreSQL debe propagarse como unavailable.

No inventar MRR, ARR, usuarios, tenants ni ingresos.

15. FASE 5L - STORAGE REAL Y FAIL-CLOSED

El commit más reciente ha añadido escritura local a public/uploads. Es útil para staging, pero no es todavía storage de producción completo.

Problemas:

1. un error de escritura solo produce warning;
2. el upload puede seguir devolviendo resultado;
3. delete devuelve true incluso si el archivo no existe;
4. la ruta de upload y delete debe ser exactamente coherente;
5. el disco local del VPS no es un sistema de backup/replicación;
6. un redeploy puede eliminar archivos si no existe volumen persistente.

En producción:

write error -> success=false

delete error -> false/error

Nunca warning + success=true.

16. FASE 5M - CHECKSUM REAL

El checksum debe ser SHA-256 de los bytes reales:

sha256(buffer)

No usar:

filename + timestamp + size

Guardar:

- checksumAlgorithm=sha256;
- checksum;
- sizeBytes;
- mimeType.

17. FASE 5N - MEDIA CONSISTENTE

El orden lógico:

1. validar archivo;
2. resolver Tenant;
3. escribir bytes en storage;
4. calcular checksum real;
5. crear MediaAsset en PostgreSQL.

Si PostgreSQL falla:

- eliminar/compensar el archivo creado.

Si storage falla:

- no crear MediaAsset.

Nunca dejar una referencia DB a un archivo inexistente.

18. FASE 5O - TENANT RESOLUTION EN MEDIA

app/api/media/route.ts

En producción el Tenant debe salir de:

- sesión;
- trusted tenant context;
- hostname verificado.

Eliminar:

tenant_demo

query tenant arbitrario

body tenant arbitrario.

19. FASE 5P - ORDER SERVICE

FENIXCMS_4 ya añadió guardas !isProductionMode() para algunos fallbacks de desarrollo:

- INITIAL_TENANTS;
- INITIAL_PRODUCTS;
- cupón FENIX10.

Esto puede mantenerse para tests locales solo si se demuestra que NODE_ENV=production impide absolutamente esas ramas.

En producción:

- DB product query failure -> error;
- DB tenant query failure -> error;
- DB coupon failure -> error;
- nunca usar datos demo.

20. FASE 5Q - NO SUPRIMIR ERRORES DE POSTGRESQL

Auditar todo el repositorio en busca de:

```
catch {}  
catch() => {}  
catch() => default  
catch() => INITIAL_*)  
console.warn(...); continue;
```

Cada caso debe clasificarse.

Si es un fallo de negocio crítico:

- relanzar;
- devolver 5xx/503;
- no continuar con datos inventados.

21. FASE 5R - PRODUCTION GUARD

Crear:

scripts/audit-production-fallbacks.ts

Debe detectar uso de:

- INITIAL_;
- tenant_demo;
- FALLBACK_;
- inMemory;
- MEMORY_;
- mock_;
- fake_;
- test_fraud;
- localhost en callbacks;
- URLs PayPal/Stripe ficticias.

El script debe diferenciar:

- archivos tests;

- documentación;
- código runtime.

Fallo si una ruta de producción contiene un fallback no permitido.

22. FASE 5S - SECRET AUDIT

Crear:

scripts/audit-production-secrets.ts

Detectar:

- passwords hardcoded;
- tokens;
- client secrets;
- DATABASE_URL usado desde componentes client;
- secretos PayPal;
- secretos Stripe;
- private keys.

El script no debe imprimir secretos encontrados; solo indicar archivo y tipo de riesgo.

23. FASE 5T - ENV AUDIT

Crear:

scripts/check-production-env.ts

En production exigir:

NODE_ENV=production
DATABASE_URL
APP_URL
PAYPAL_CLIENT_ID
PAYPAL_CLIENT_SECRET
PAYPAL_WEBHOOK_ID
PAYPAL_BASE_URL
STORAGE_PROVIDER o configuración de storage persistente

Validar:

- APP_URL HTTPS;
- PAYPAL_MODE=live;
- PAYPAL_BASE_URL=https://api-m.paypal.com;
- no localhost;
- no sandbox.

24. FASE 5U - SOURCE OF TRUTH CONTRACT

SaaS:

PostgreSQL + PayPal.

Store:

PostgreSQL + payment provider real.

Auth:

PostgreSQL + PasswordService + SessionService.

Media:

PostgreSQL + persistent storage.

Admin metrics:

PostgreSQL.

No memoria ni initialState como fuente de verdad comercial.

25. FASE 5V - PAYMENT FLOW FINAL

Debe quedar:

Application DB

-> Plan DB

-> CheckoutSession

-> PayPal Order

-> approval

-> capture

-> validate COMPLETED

-> amount match

-> currency match

-> reference match

-> SaaSProvisioningService

-> PostgreSQL transaction

-> completed.

No:

INITIAL_*

No Tenant before payment.

No fake transaction.

No memory success.

26. FASE 5W - ORDER PAYMENT

El checkout de tienda debe mantener:

- precio server-side;
- stock server-side;
- tenant server-side;
- cupón server-side;
- idempotency.

El Payment de tienda debe utilizar:

paymentType=ORDER_PAYMENT.

No mezclarlo con:

SAAS_LICENSE.

27. FASE 5X - WEBHOOK PAYPAL

Mantener:

- raw body;
- verify-webhook-signature;
- webhook ID;
- eventId unique;
- idempotencia persistida.

Si PostgreSQL falla al persistir webhook:

- no marcar processed;
- permitir retry/reconciliation.

No MEMORY_WEBHOOK_EVENTS como fuente de verdad en producción.

28. FASE 5Y - HEALTH READINESS

/api/health debe separar:

Liveness:

proceso Node funciona.

Readiness:

- PostgreSQL accesible;
- configuración PayPal válida;
- storage disponible/configurado.

DB caída -> readiness 503.

29. FASE 5Z - PRODUCTION GATE CI

Crear:

.github/workflows/production-gate.yml

Ejecutar:

```
npm ci
npx prisma generate
npm run lint
npm test
npm run build
npm run audit:production
```

El workflow de documentación/PDF no debe ser la única Action del proyecto.

30. FASE 5AA - TESTS ESPECÍFICOS

Crear:

tests/fenixcms-fase5.test.ts

P01 Application DB failure -> no INITIAL fallback.
P02 Plan DB failure -> no INITIAL fallback.
P03 Checkout -> no Tenant creation.
P04 Payment create failure -> no FALLBACK_PAYMENTS.
P05 Payment update failure -> no fake COMPLETED.
P06 Auth DB failure -> no memory user in production.
P07 SuperAdmin DB failure -> no demo metrics.
P08 Media write failure -> no success.
P09 Media tenant resolution -> no tenant_demo.
P10 Order product DB failure -> no INITIAL_PRODUCTS.
P11 Coupon DB failure -> no FENIX10.
P12 production env missing -> audit fails.
P13 hardcoded password -> audit fails.
P14 fake provider URL -> audit fails.
P15 PayPal COMPLETED + DB failure -> recoverable state.
P16 recovery -> exactly one Payment/Tenant/License/Subscription/Invoice.
P17 DB empty -> empty state, not demo data.
P18 DB unavailable -> service unavailable.
P19 storage checksum = real SHA-256.
P20 concurrent provisioning -> exactly one result.

31. FASE 5AB - PRUEBA DE DB VACÍA

Con PostgreSQL funcionando y sin registros de negocio:

- Applications [];
- Plans [];
- Tenants [];
- Payments [];
- Users [];
- Products [].

La aplicación debe mostrar estados vacíos.

Nunca INITIAL_*.

32. FASE 5AC - PRUEBA DB CAÍDA

Con PostgreSQL no disponible:

- login -> error;
- checkout -> error;

- capture -> error/recoverable;
- admin -> unavailable;
- orders -> error;
- media -> error;
- readiness -> 503.

Nunca:

- fake users;
- fake tenants;
- fake payments;
- fake metrics.

33. FASE 5AD - PRUEBA STORAGE FALLIDO

Simular writeFile/storage provider error.

Resultado:

- success=false;
- no MediaAsset válido;
- no URL falsa;
- error auditable.

34. FASE 5AE - PRUEBA PAYPAL FALLIDO

PayPal createOrder timeout/error:

- CheckoutSession FAILED/retryable;
- no Tenant;
- no License;
- no Subscription;
- no Invoice.

35. FASE 5AF - PAYPAL COMPLETED + DB FAIL

PayPal:

COMPLETED

PostgreSQL:

timeout

Resultado:

- no second charge;
- evidence of payment retained;
- checkout recoverable;
- no final duplicate provisioning.

Cuando DB vuelva:

reconcile -> provision once.

36. FASE 5AG - PRODUCTION SECRETS

Los secretos deben existir solamente en el entorno del servidor.

Nunca:

- components client;
- localStorage;
- public;
- initialData;
- Git.

37. FASE 5AH - PACKAGE SCRIPT

Añadir:

```
`audit:production`: `tsx scripts/audit-production-fallbacks.ts && tsx scripts/audit-production-secrets.ts && tsx scripts/check-production-env.ts`
```

Integrar con CI.

38. FASE 5AI - ARCHIVOS PRINCIPALES

Revisar:

```
lib/services/payment.service.ts
lib/services/saas-checkout.service.ts
lib/services/super-admin.service.ts
lib/services/auth.service.ts
lib/services/webhook.service.ts
lib/storage/storage.service.ts
lib/services/order.service.ts
lib/prisma.ts
app/api/billing/checkout/route.ts
app/api/billing/capture/route.ts
app/api/tenants/provision/route.ts
app/api/media/route.ts
app/api/orders/route.ts
prisma/schema.prisma
package.json
scripts/run-all-tests.ts
```

Crear:

```
tests/fenixcms-fase5.test.ts
scripts/audit-production-fallbacks.ts
scripts/audit-production-secrets.ts
scripts/check-production-env.ts
.github/workflows/production-gate.yml
```

39. FASE 5AJ - PROMPT MAESTRO PARA AI STUDIO

INICIO FASE FENIXCMS_5

DONDE SE EJECUTA:

AI STUDIO / repositorio fenixcmssl-spec/moreno

VALIDACIÓN:

ORDENADOR DEBIAN

NO DESPLEGAR:

No desplegar al VPS producción hasta completar todas las pruebas.

OBJETIVO:

Hacer que FenixCMS sea fail-closed en producción.

PostgreSQL es la única fuente de verdad del negocio.

No se permite sustituir DB por initialData, memoria, datos demo o valores ficticios.

PASO 1 - AUDITORÍA

Audita:

- PaymentService
- SaaSCheckoutService
- TenantService
- AuthService
- SuperAdminService
- WebhookService
- StorageService
- OrderService
- Media API
- Billing APIs
- Provision API
- Health

- Prisma

Busca:

INITIAL_

FALLBACK_

inMemory

MEMORY_

tenant_demo

fake_

mock_

test_fraud

success=true después de catch

catch{}

catch() => {}

valores monetarios por defecto

passwords hardcoded

URLs PayPal/Stripe ficticias

Primero enumera los hallazgos y después modifica.

PASO 2 - PAYMENT SERVICE

Modificar lib/services/payment.service.ts.

En producción:

- no INITIAL_APPLICATIONS;
- no INITIAL_PLANS;
- no INITIAL_TENANTS;
- no FALLBACK_PAYMENTS;
- no precio/moneda por defecto;
- no ensureTenantExists durante create checkout.

Si Application o Plan no existe en PostgreSQL:

rechazar.

Si PostgreSQL falla:

rechazar.

PASO 3 - CHECKOUT

Create checkout:

1. validar Application;
2. validar Plan;
3. validar relación;
4. calcular precio;
5. crear CheckoutSession;
6. crear PayPal Order;
7. guardar paypalOrderId;
8. devolver approvalUrl.

NO crear Tenant.

PASO 4 - PROVISIONING

Después de Capture COMPLETED y validaciones:

- amount;
- currency;
- reference;
- captureId único.

Ejecutar transacción:

Payment

Tenant

User

Membership

License

Subscription

Invoice

Domain

CheckoutSession COMPLETED

Todo idempotente.

PASO 5 - PAYMENT FAIL-CLOSED

No convertir fallos de PostgreSQL en:

- COMPLETED;
- memory payment;
- fake success.

PASO 6 - AUTH

Todas las funciones de AuthService deben ser PostgreSQL-only en production.

Eliminar fallback de memoria en producción.

Eliminar passwords maestras.

PASO 7 - SUPER ADMIN

DB vacía -> empty state.

DB caída -> unavailable.

Nunca INITIAL_* en runtime de production.

PASO 8 - STORAGE

Storage debe ser persistente.

El fallo de escritura debe devolver error.

Checksum real SHA-256.

MediaAsset solo se crea si el archivo existe.

No success=true tras excepción.

PASO 9 - MEDIA

No tenant_demo.

Tenant desde session/trusted context/verified hostname.

PASO 10 - ORDER

Mantener las protecciones de FENIXCMS_4.

Production fallback = completamente bloqueado.

PASO 11 - AUDITORÍAS

Crear:

scripts/audit-production-fallbacks.ts

scripts/audit-production-secrets.ts

scripts/check-production-env.ts

PASO 12 - CI

Crear production-gate.yml:

npm ci

npx prisma generate

npm run lint

npm test

npm run build

npm run audit:production

PASO 13 - TESTS

Crear tests/fenixcms-fase5.test.ts con:

- DB failure;
- empty DB;
- demo fallback blocked;
- payment fallback blocked;
- auth fallback blocked;
- metrics fallback blocked;
- media failure;
- PayPal failure;
- PayPal completed + DB failure;
- recovery;
- concurrency;
- secrets;
- fake URLs.

PASO 14 - VALIDACIÓN

Ejecutar:

npm ci

npx prisma generate

npm run lint
npm test
npm run build
npm run audit:production

Devuelve:

1. archivos creados;
2. archivos modificados;
3. migraciones;
4. tests reales ejecutados;
5. resultado exacto de lint;
6. resultado exacto de build;
7. resultado exacto de audit:production;
8. blockers restantes.

No llames PASS a un test que haya utilizado fallback para simular producción.

FIN FASE FENIXCMS_5

40. CRITERIOS DE TERMINADO

FENIXCMS_5 NO está terminada hasta demostrar:

- ☐ SaaS checkout no usa INITIAL_* en production.
- ☐ SaaS checkout no crea Tenant antes del pago.
- ☐ Application/Plan salen de PostgreSQL.
- ☐ Payment no usa memoria en production.
- ☐ Payment no se marca COMPLETED si PostgreSQL falla.
- ☐ Auth no utiliza fallback de usuarios en production.
- ☐ Super Admin no muestra métricas demo en production.
- ☐ Media no usa tenant_demo.
- ☐ Storage no devuelve éxito después de fallo.
- ☐ Checksum es de bytes reales.
- ☐ Orders no usan datos demo en production.
- ☐ Production fallback audit pasa.
- ☐ Secret audit pasa.
- ☐ Environment audit pasa.
- ☐ Tests FENIXCMS_5 pasan.
- ☐ npm run lint pasa.
- ☐ npm run build pasa.
- ☐ npm test pasa.
- ☐ GitHub production-gate pasa.

41. DECISIÓN DE PRODUCCIÓN

El mayor riesgo encontrado actualmente no es una función visual del CMS.

Es la posibilidad de que una ruta de negocio continúe funcionando con una realidad alternativa cuando PostgreSQL o un dato real no está disponible.

La arquitectura de producción debe quedar:

PostgreSQL
+
PayPal real
+
Storage persistente
+
Webhooks verificados
+
Fail-closed
+
Idempotencia
+
Auditoría

y nunca:

PostgreSQL

+

INITIAL_*

+

Memory

+

Demo.

FIN FASE 5